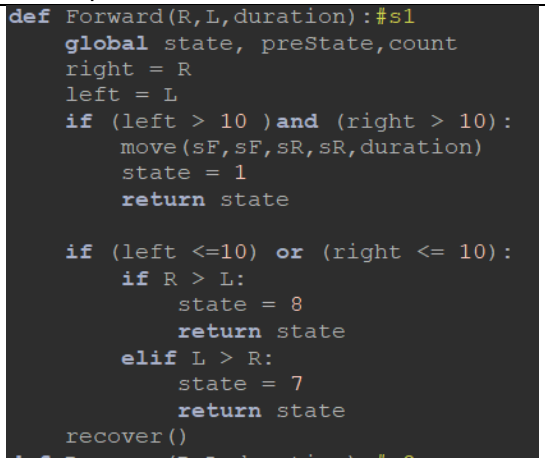
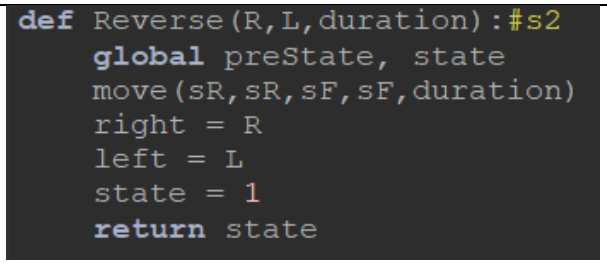
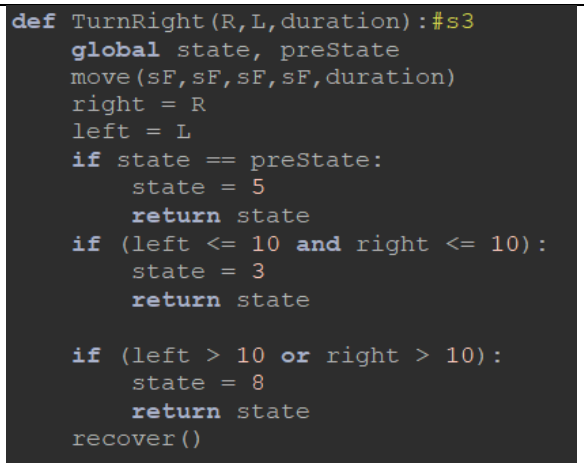
State	Description
Forward	 The forward state is the starting state in this augmented finite state machine (AFSM). The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. If the distance of both sensors is > than 10, the robot will move forwards, and it will go to the forward state again. If either sensor gives a distance of <= 10, if the right sensor is > the left, it transitions to Veer Right state. If the right sensor is < the left, it transitions to the Veer Left state.
Reverse	 The reverse state can only be transitioned to from the recovery state. The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. This state causes the robot to move backwards. Once the duration is exceeded, the AFSM will transition to the forward state.
Turn Right	 The turn right state can be transitioned to from the Veer Right state. The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. This state causes the robot to rotate 90 degrees to the right. If the left and right sensors give a value of <= 10, the AFSM will loop back to the turn right state. After

	looping back, the AFSM will transition to the flip state. If the left or right sensors give a value of > 10, the AFSM will transition to the Veer Right state.

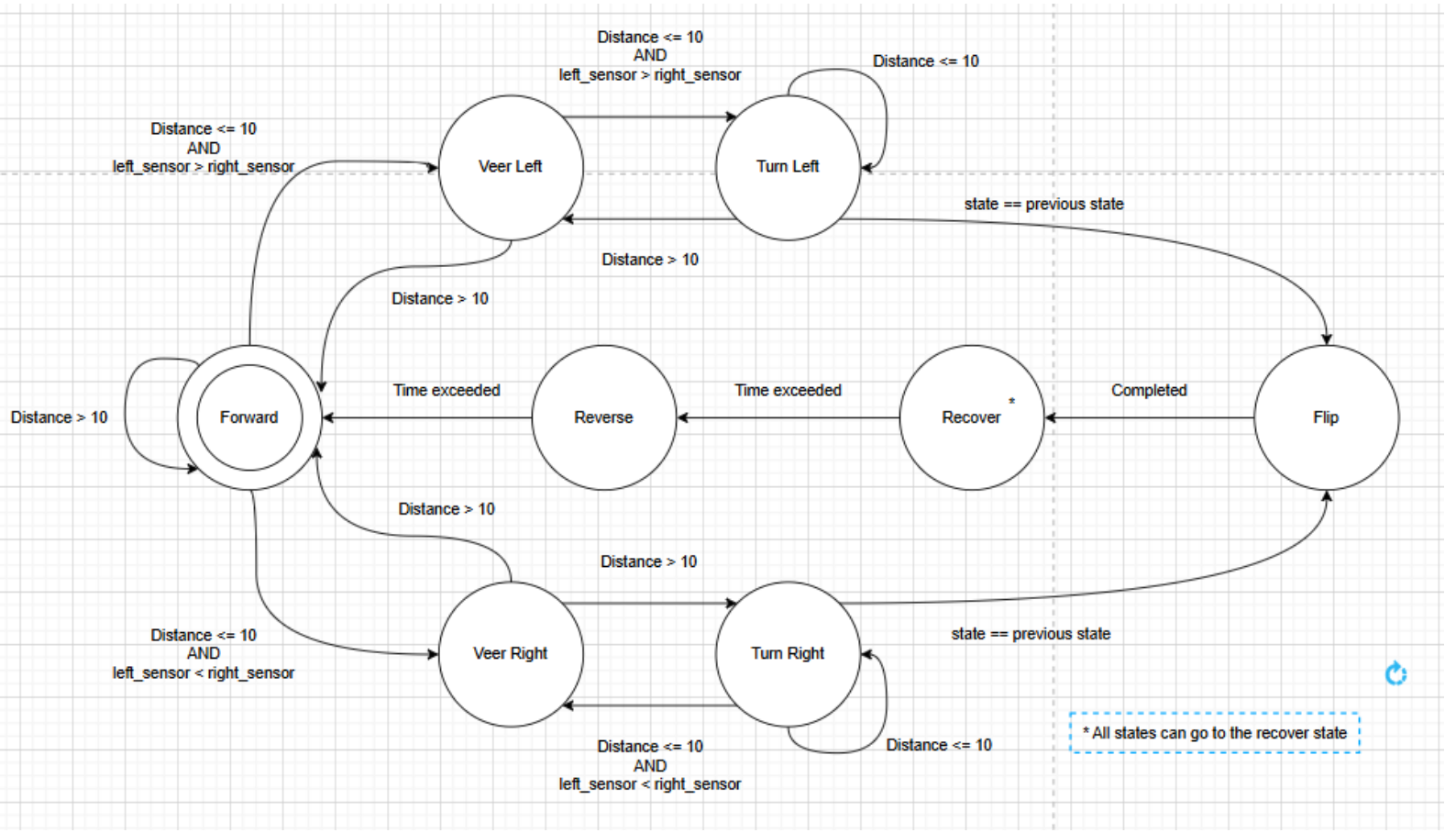
Turn Left	`def TurnLeft(R,L,duration):#s4`
	`  global state,preState`
	`  move(sR,sR,sR,sR,duration)`
	`  right = R`
	`  left = L`
	`  if state == preState:`
	`  state = 5`
	`  return state`
	`  if (left <= 10 and right <= 10) :`
	`  state = 4`
	`  return state`
	`  if left > 10 or right > 10:`
	`  state = 7`
	`  return state`
	`  recover()`
	The turn left state can be transitioned to from the Veer left state. The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. This state causes the robot to rotate 90 degrees to the left. If the left and right sensors give a value of <= 10, the AFSM will loop back to the turn left state. After looping back, the AFSM will transition to the flip state. If the left or right sensors give a value of > 10, the AFSM will transition to the Veer left state.

Flip	`def flip(duration):#s5`
	`  global state, preState`
	`  move(sR,sR,sR,sR,duration)`
	`  state = 6`
	`  return state`
	The flip state can be transitioned to from the turn right state and the turn left state. The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. This state causes the robot to rotate 180 degrees. This state acts as a way to get the robot “unstuck” from a corner, if it repeats the same turn twice.

Recover	`def recover(duration):#s6`
	`  global state`
	`  move(0,0,0,0,duration)`
	`  state = 2`
	`  return state`
	The recover state can be transitioned to from any state in the AFSM. The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. If any state transitions to itself 3 times in a row, the recovery state will be transitioned to. This will also be transitioned to if no transition conditions are met in a state. It causes the robot to stop moving for a specified duration before transitioning to the reverse state.

Veer Left	```
	def veerLeft(R,L,duration):#s7
	    global state, preState
	    move(sF,sF,int(sR*0.5),int(sR*0.5),duration)
	    right = R
	    left = L
	    if((left <= 10) and (right <= 10)):
	        if L>R:
	            state = 4
	            return state
	    if(left > 10) or (right > 10):
	        state = 1
	        return state
	``
	The Veer Left state can be transitioned to from the forward state and the turn left state. The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. This state causes the robot to move forwards, to the left. This enables the robot to "see" if there is a corner or turning in its path. The forward-left movement is accomplished by halving the forwards power of the left servos. If the left and right sensors give a value of <= 10, if left > right, the AFSM transitions to the turn left state. If either sensor gives a value of > 10, the AFSM transitions to the forward state.

Veer Right	```
	def veerRight(R,L,duration):#s8
	    global state, preState
	    move(int(sF*0.5),int(sF*0.5),sR,sR,duration)
	    right = R
	    left = L
	    if((left <= 10) and (right <= 10)):
	        if R > L:
	            state = 3
	            return state
	    if(left >10) or (right >10):
	        state = 1
	        return state
	``
	The Veer Right state can be transitioned to from the forward state and the turn right state. The left and right sensor values are passed into the function, with a specified duration, which determines how long it will remain in this state. This state causes the robot to move forwards, to the right. This enables the robot to "see" if there is a corner or turning in its path. The forward-right movement is accomplished by halving the forwards power of the right servos. If the left and right sensors give a value of <= 10, if right > left, the AFSM transitions to the turn right state. If either sensor gives a value of > 10, the AFSM transitions to the forward state.


Distance <= 10
AND
left_sensor > right_sensor
Distance <= 10
Distance <= 10
AND
left_sensor > right_sensor
Veer Left
Turn Left
state == previous state
Distance > 10
Distance > 10
Distance > 10
Time exceeded
Time exceeded
Completed
Forward
Reverse
Recover
*
Flip
Distance > 10
Distance > 10
Distance > 10
Distance <= 10
AND
left_sensor < right_sensor
Veer Right
Turn Right
state == previous state
Distance <= 10
AND
left_sensor < right_sensor
Distance <= 10
* All states can go to the recover state